Django Authentication System — Detailed Notes

A. Django Authentication System Overview

1. What is Authentication?

Authentication is the process of verifying the identity of a user.

It determines whether a person accessing an application is actually the person they claim to be.

For example, when a user logs into a website using a username and password, Django checks whether the provided credentials are valid.

Example:

- Username: rashik
- Password: MyPassword123

Django verifies these credentials against the information stored in its database.

If the credentials are valid, the user is authenticated.

2. Authentication vs Authorization

These are two important concepts in web application security.

Authentication:

- Determines who the user is.
- Verifies the user's identity.
- Usually involves usernames, passwords, tokens or other credentials.

Authorization:

- Determines what an authenticated user is allowed to do.
- Controls access to pages, resources and operations.
- Uses permissions, groups and access-control rules.

Example:

Consider a student management application.

User	Authentication	Authorization
Student	Logs in successfully	Views their profile
Teacher	Logs in successfully	Views and updates student records
Admin	Logs in successfully	Manages all records

Authentication verifies the identity; authorization determines the allowed actions.

3. Django's Built-in Authentication System

Django provides a built-in authentication framework through `django.contrib.auth`.

It provides several features:

1. User account management.
2. Password hashing and verification.
3. Login and logout functionality.
4. Authentication backends.
5. Session management.
6. Permissions and groups.
7. Password reset functionality.

This means developers do not need to implement an entire authentication system manually.

4. Important Django Authentication Components

Component	Purpose
User model	Stores user account information
`authenticate()`	Verifies user credentials
`login()`	Logs in a user
`logout()`	Logs out a user
Authentication backend	Defines how users are authenticated
Authentication middleware	Makes the current user available in requests
Session framework	Maintains user sessions

Permissions	Controls access to operations
Groups	Organizes users with shared permissions

5. How Django Authentication Works

The general authentication process:

1. A user submits a login form.
2. Django receives the username and password.
3. `authenticate()` checks the supplied credentials.
4. The configured authentication backend verifies them.
5. If valid, Django returns the User object.
6. `login()` associates the user with the current session.
7. Django sends a session cookie to the browser.
8. On subsequent requests, Django identifies the logged-in user using the session.

Authentication flow:

Browser → Login Form → `authenticate()` → Authentication Backend → User Database → `login()` → Session Created → Authenticated User.

6. Required Django Settings

Django's default project configuration normally includes these applications:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

Important applications:

- `django.contrib.auth`: Provides user authentication and permissions.
- `django.contrib.contenttypes`: Supports the content type framework used by permissions.
- `django.contrib.sessions`: Provides session management.

Important middleware:

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
]
```

SessionMiddleware: Makes session data available through `request.session`.

AuthenticationMiddleware: Makes the current user available through `request.user`.

AuthenticationMiddleware depends on SessionMiddleware being installed before it.

B. User Model and Authentication Backends

1. What is the User Model?

A User model represents a registered account in a Django application.

Django provides a built-in User model:

from django.contrib.auth.models import User

It stores information about registered users.

2. Important Fields of the Built-in User Model

Field	Description
`id`	Unique identifier for the user
`username`	User's username
`first_name`	User's first name
`last_name`	User's last name
`email`	User's email address
`password`	Hashed password

`is_active`	Determines whether the account is active
`is_staff`	Determines whether the user can access the Django admin site
`is_superuser`	Grants all permissions
`last_login`	Last login timestamp
`date_joined`	Account creation timestamp

3. User Model and Database

The default User model is stored in the `auth_user` database table.

When a new user registers, Django creates a record in this table.

Example:

```
from django.contrib.auth.models import User

user = User.objects.create_user(
    username='rashik',
    email='rashik@example.com',
    password='StrongPassword123'
)
```

This creates a user account.

To retrieve all users:

```
users = User.objects.all()
```

To retrieve a particular user:

```
user = User.objects.get(username='rashik')
```

To retrieve users based on a condition:

```
users = User.objects.filter(is_active=True)
```

4. Creating Users: `create()` vs `create_user()`

These two methods behave differently.

Using `create_user()`:

```
User.objects.create_user(
   username='student1',
   password='Student@123'
)
```

- Creates a user account.
- Automatically hashes the password.
- Uses Django's password-setting mechanism.

Using `create()`:

```
User.objects.create(
   username='student2',
   password='Student@123'
)
```

- Creates a database record.
- Does not automatically hash the password assigned directly to the password field.
- Can result in an unusable login password.

Always use `create_user()` or `set_password()` when creating user accounts manually.

5. Password Hashing

Password hashing is an important security feature.

Django does not normally store passwords in plain text.

Instead, it stores password hashes using a configured password-hashing algorithm.

Example:

```
user = User.objects.create_user(
   username='rashik',
   password='Hello123'
)
```

The database stores a salted password hash, not the original password.

To check whether a password is correct:

```
user.check_password('Hello123')
```

Returns:

- `True`: Password is correct.
- `False`: Password is incorrect.

To change a password:

```
user.set_password('NewPassword123')
user.save()
```

Never assign a raw password directly to `user.password`.

6. What are Authentication Backends?

Authentication backends define how Django verifies a user's identity.

They determine which authentication mechanism is used to validate credentials.

For example:

- Username and password authentication.
- Email-based authentication.
- Authentication against an external service.
- Custom authentication logic.

Django uses the following backend by default:

'django.contrib.auth.backends.ModelBackend'

7. ModelBackend

`ModelBackend` is Django's default authentication backend.

It generally performs these operations:

1. Receives the username and password.
2. Retrieves the corresponding user.
3. Checks the password hash.
4. Checks whether the account is active.
5. Returns the User object if authentication succeeds.

If authentication fails, it returns `None`.

8. Configuring Authentication Backends

In `settings.py`:

```
AUTHENTICATION_BACKENDS = [
    'django.contrib.auth.backends.ModelBackend',
]
```

Django uses ModelBackend by default, so explicitly writing this setting is optional unless you want to customize the backend configuration.

9. The `authenticate()` Function

`authenticate()` checks whether the supplied credentials are valid.

```
from django.contrib.auth import authenticate

user = authenticate(
    request,
    username='rashik',
    password='Hello123'
)
```

Possible results:

- Returns a User object if authentication succeeds.
- Returns `None` if authentication fails.

Important: `authenticate()` does not log the user in.

It only verifies the credentials.

10. Custom Authentication Backend

Sometimes an application needs authentication using email instead of username.

A custom backend can implement this.

Example:

```
from django.contrib.auth.backends import ModelBackend
from django.contrib.auth import get_user_model

class EmailBackend(ModelBackend):

    def authenticate(
        self,
        request,
        username=None,
```

```
        password=None,
        **kwargs
    ):
        User = get_user_model()

        try:
            user = User.objects.get(email=username)
        except User.DoesNotExist:
            return None

        if (
            user.check_password(password)
            and self.user_can_authenticate(user)
        ):
            return user

        return None
```

Configure it:

```
AUTHENTICATION_BACKENDS = [
    'your_app.backends.EmailBackend',
]
```

Replace `your_app` with the actual application name.

This is a basic example. A production implementation should also handle duplicate email addresses, database lookup errors and account-enumeration concerns.

Important: A custom authentication backend changes how credentials are verified. It does not automatically change the login form's fields.

11. Custom User Model

Django also allows developers to create a custom User model.

Reasons include:

- Using email instead of username as the primary login identifier.
- Adding extra user information.
- Changing user-related fields.
- Implementing application-specific account requirements.

For example:

```
from django.contrib.auth.models import AbstractUser
```

```
from django.db import models

class CustomUser(AbstractUser):

    phone_number = models.CharField(
        max_length=15,
        blank=True
    )
```

Then configure:

```
AUTH_USER_MODEL = 'your_app.CustomUser'
```

For new projects, deciding on the user model before the first database migrations is strongly recommended. Changing it after a project has existing migrations and user data can be complex.

Use `get_user_model()` in application code when you want to work with whichever User model is configured.

C. Creating User Registration and Login Forms

1. What is User Registration?

Registration is the process of creating a new user account.

A typical registration form collects:

- Username.
- Email address.
- Password.
- Password confirmation.

Django provides a built-in form called `UserCreationForm`.

Import:

```
from django.contrib.auth.forms import UserCreationForm
```

2. UserCreationForm

`UserCreationForm` is a built-in Django form designed to create new users.

It provides:

1. Username field.
2. Password field.
3. Password confirmation field.
4. Password validation.
5. Username validation.
6. Password hashing when saving the user.

Example:

```
from django.contrib.auth.forms import UserCreationForm

form = UserCreationForm()
```

This creates an empty registration form.

3. Creating a Registration View

In `views.py`:

```
from django.shortcuts import render, redirect
from django.contrib.auth.forms import UserCreationForm

def register(request):

    if request.method == 'POST':

        form = UserCreationForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('login')

    else:

        form = UserCreationForm()

    return render(
        request,
        'register.html',
        {'form': form}
    )
```

Explanation:

`request.method == 'POST'`

Checks whether the user submitted the form.

`UserCreationForm(request.POST)`

Receives the submitted form data.

`form.is_valid()`

Validates the username, password and password confirmation.

`form.save()`

Creates the user account and hashes the password.

`redirect('login')`

Redirects the user to the login page after successful registration.

`UserCreationForm` does not include an email field by default. An email field can be added using a custom form based on `UserCreationForm`.

4. Registration Template

Create `register.html`:

```
<h2>Register</h2>

<form method="POST">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">
        Register
    </button>

</form>
```

Explanation:

- `method="POST"` submits form data using POST.
- `{% csrf_token %}` protects against cross-site request forgery.
- `{{ form.as_p }}` displays form fields as paragraphs.
- The submit button sends the registration information.

5. What is AuthenticationForm?

`AuthenticationForm` is Django's built-in login form.

Import:

from django.contrib.auth.forms import AuthenticationForm

It provides username and password fields and validates the credentials through Django's authentication system.

6. Creating a Login View

In `views.py`:

```
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth import login
from django.shortcuts import render, redirect

def user_login(request):

    if request.method == 'POST':

        form = AuthenticationForm(
            request,
            data=request.POST
        )

        if form.is_valid():

            user = form.get_user()

            login(request, user)

            return redirect('home')

    else:

        form = AuthenticationForm()
```

```
    return render(
        request,
        'login.html',
        {'form': form}
    )
```

Explanation:

`AuthenticationForm(request, data=request.POST)`

Receives the submitted username and password.

`form.is_valid()`

Checks whether the credentials are valid.

`form.get_user()`

Returns the authenticated User object after successful validation.

`login(request, user)`

Associates the user with the current session.

`redirect('home')`

Redirects the user to the home page.

7. Login Template

Create `login.html`:

```
<h2>Login</h2>

<form method="POST">

    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">
        Login
    </button>
```

</form>

8. Logout View

Django provides a built-in `logout()` function.

```
from django.contrib.auth import logout
from django.shortcuts import redirect

def user_logout(request):

    logout(request)

    return redirect('login')
```

This logs out the current user and clears the authenticated session.

For a production application, use POST requests for logout and protect them with CSRF validation.

9. URL Configuration

In your application's `urls.py`:

```
from django.urls import path
from . import views

urlpatterns = [

    path(
        'register/',
        views.register,
        name='register'
    ),

    path(
        'login/',
        views.user_login,
        name='login'
    ),

    path(
        'logout/',
        views.user_logout,
        name='logout'
```

```
    ),
]
```

Include these application URLs in the main project's URL configuration.

10. Registration vs Login

Registration	Login
Creates a new account	Verifies an existing account
Uses UserCreationForm	Uses AuthenticationForm
Saves a new User object	Establishes a session
Uses `form.save()`	Uses `login(request, user)`

D. Managing User Sessions and Permissions

PART 1: SESSION MANAGEMENT

1. What is a Session?

A session is a mechanism that allows a web application to remember information about a user across multiple requests.

HTTP is a stateless protocol.

This means each HTTP request is independent by default. A server does not automatically remember the previous request.

Sessions help Django maintain user-related information between requests.

Example:

1. User logs in.
2. User opens the dashboard.
3. User visits the profile page.
4. User visits the settings page.

Django remembers the authenticated user throughout these requests using the session.

2. How Django Sessions Work

The process:

1. User submits login credentials.
2. Django authenticates the user.
3. `login()` associates the user with the current session.
4. Django saves session data on the server.
5. Browser receives a session cookie.
6. Browser sends that cookie with subsequent requests.
7. Django identifies the session and retrieves the user.

Important: The session cookie normally contains a session identifier, not the user's password.

3. SessionMiddleware

Django provides:

'django.contrib.sessions.middleware.SessionMiddleware'

It enables session functionality.

It makes session information accessible through:

request.session

4. AuthenticationMiddleware

Django provides:

'django.contrib.auth.middleware.AuthenticationMiddleware'

It associates the current request with the authenticated user.

This makes it possible to access:

request.user

For an authenticated user, this returns the User object.

For an anonymous visitor, it returns an AnonymousUser object.

5. The `login()` Function

The login function associates an authenticated user with the current session.

from django.contrib.auth import login

login(request, user)

After successful login:

request.user

represents the logged-in user.

6. The `logout()` Function

The logout function ends the current authenticated session.

from django.contrib.auth import logout

logout(request)

After logout:

request.user.is_authenticated

returns `False` for the logged-out request.

7. The `request.session` Object

Django also allows developers to store custom data in sessions.

Example:

```
def home(request):
    request.session['username'] = 'rashik'
    return render(request, 'home.html')
```

Retrieve the value:

username = request.session.get('username')

Delete a value:

del request.session['username']

Clear all session data:

request.session.flush()

`flush()` clears the current session data and session cookie.

Use `logout()` for normal user logout because it also performs Django's authentication-specific logout handling.

8. Session Database

By default, Django commonly uses database-backed sessions.

The session records are stored in:

django_session

You can inspect session records through Django's database.

9. Important Session Settings

SESSION_COOKIE_AGE = 1209600

SESSION_COOKIE_HTTPONLY = True

SESSION_COOKIE_SECURE = True

SESSION_EXPIRE_AT_BROWSER_CLOSE = False

Setting	Purpose
`SESSION_COOKIE_AGE`	Session cookie lifetime in seconds
`SESSION_COOKIE_HTTPONLY`	Prevents JavaScript from accessing the cookie
`SESSION_COOKIE_SECURE`	Sends the cookie only over HTTPS
`SESSION_EXPIRE_AT_BROWSER_CLOSE`	Controls browser-close cookie expiry

`1209600` seconds equals 14 days.

For local HTTP development, `SESSION_COOKIE_SECURE` should generally be `False`. Enable it when the application is served over HTTPS.

PART 2: PERMISSION MANAGEMENT

10. What are Permissions?

Permissions determine what actions a user can perform within an application.

For example, in a student management system:

- Students can view their own information.
- Teachers can view and update student records.
- Administrators can manage all records.

Django provides a built-in permission framework.

11. Default Model Permissions

When you create a Django model and run migrations, Django automatically creates four default permissions for that model.

Example:

```
class Student(models.Model):

    name = models.CharField(max_length=100)

    age = models.IntegerField()
```

Django generates:

Permission	Purpose
`add_student`	Add a student
`view_student`	View students
`change_student`	Modify student information
`delete_student`	Delete students

These permissions are associated with the model's content type.

12. Checking Permissions

Use the `has_perm()` method.

request.user.has_perm('api.view_student')

Returns:

- `True` if the user has the permission.
- `False` otherwise.

Here:

- `api` is the application label.
- `view_student` is the permission codename.

13. Assigning Permissions to Users

Import:

from django.contrib.auth.models import User, Permission

Retrieve a user:

user = User.objects.get(username='rashik')

Retrieve a permission:

```
permission = Permission.objects.get(
    codename='view_student'
)
```

Assign the permission:

user.user_permissions.add(permission)

Now the user has that permission, assuming the user is active and no other access rules prevent the operation.

14. Removing Permissions

user.user_permissions.remove(permission)

This removes the directly assigned permission.

To remove all directly assigned permissions:

user.user_permissions.clear()

15. What are Groups?

Groups are collections of users that share the same permissions.

Instead of assigning permissions to 50 teachers individually, you can create a Teachers group and assign permissions to that group.

All users in the group inherit its permissions.

16. Creating Groups

```
from django.contrib.auth.models import Group

teachers = Group.objects.create(
    name='Teachers'
)
```

17. Assigning Permissions to Groups

```
from django.contrib.auth.models import Permission

permissions = Permission.objects.filter(
    codename__in=[
        'view_student',
        'change_student'
    ],
    content_type__app_label='api'
)

teachers.permissions.add(*permissions)
```

18. Adding Users to Groups

```
user = User.objects.get(
    username='rashik'
)

user.groups.add(teachers)
```

Now the user inherits the permissions assigned to the Teachers group.

Check the permission:

```
user.has_perm('api.view_student')
```

19. Removing Users from Groups

```
user.groups.remove(teachers)
```

Remove all group memberships:

```
user.groups.clear()
```

20. Superusers and Staff Users

Django provides two important User fields.

is_staff:

```
user.is_staff = True
user.save()
```

Allows the user to access the Django admin site, subject to admin permissions.

is_superuser:

```
user.is_superuser = True
user.save()
```

Grants all permissions without needing individual assignments.

A superuser is generally used for full administrative access.

Do not confuse `is_staff` with `is_superuser`. They serve different purposes.

21. Restricting Pages with login_required

Django provides the `login_required` decorator.

```
from django.contrib.auth.decorators import login_required

@login_required
def dashboard(request):

    return render(
        request,
        'dashboard.html'
    )
```

If a user is not authenticated, Django redirects them to the configured login URL.

Configure:

```
LOGIN_URL = 'login'

LOGIN_REDIRECT_URL = 'home'
```

LOGOUT_REDIRECT_URL = 'login'

22. Restricting Pages with permission_required

Django also provides `permission_required`.

```
from django.contrib.auth.decorators import permission_required

@permission_required('api.delete_student')
def delete_student(request):

    # Delete student logic

    pass
```

Only users with the required permission can access the view.

By default, a user without permission is redirected to the login URL. To return a 403 Forbidden response instead:

```
@permission_required(
    'api.delete_student',
    raise_exception=True
)
def delete_student(request):
    pass
```

23. User Groups vs Permissions

Groups	Permissions
Organize users into categories	Define allowed actions
Can contain multiple users	Can be assigned to users or groups
Inherit assigned permissions	Control access to resources and operations
Example: Teachers	Example: `change_student`

24. Object-Level Permissions

Object-level permissions determine whether a user can access a particular object.

Example:

A student should only be able to update their own student record.

Django's standard ModelBackend does not automatically implement per-object permission assignment. Object-level access rules can be implemented using custom view logic or an appropriate object-permission package.

Example custom check:

```
def has_object_permission(request, obj):

    return obj.user == request.user
```

This assumes the Student model has a relationship to the User model.

---

5. Complete Authentication Flow

A typical Django authentication application works as follows:

Registration:

1. User opens the registration page.
2. User submits the registration form.
3. Django validates the submitted data.
4. UserCreationForm creates the User object.
5. Django hashes the password.
6. User account is saved in the database.

Login:

1. User opens the login page.
2. User enters username and password.
3. AuthenticationForm validates the credentials.
4. Django's authentication backend verifies the user.
5. `login()` establishes the authenticated session.
6. User is redirected to the home page.

Protected page:

1. User requests a protected page.
2. AuthenticationMiddleware identifies the current user.
3. `login_required` checks authentication.
4. Permission checks can determine whether the user can perform the action.
5. Access is granted or denied.

Logout:

1. User sends a logout request.
2. Django executes `logout()`.
3. The authenticated session is cleared.
4. User becomes anonymous.

---

6. Important Security Practices

1. Never store passwords in plain text.
2. Use Django's `create_user()` or `set_password()`.
3. Always include CSRF protection in browser-based POST forms.
4. Use HTTPS in production.
5. Protect private pages with authentication checks.
6. Use permissions to restrict sensitive operations.
7. Never trust frontend restrictions alone; validate access on the server.
8. Use POST requests for state-changing operations such as logout and deletion.
9. Do not expose sensitive user information through APIs.
10. Use secure cookie settings in production.
11. Avoid giving every user superuser privileges.
12. Validate user input before saving it.

---

7. Important Django Authentication Functions and Methods

Function / Method	Purpose
`authenticate()`	Verifies credentials
`login()`	Logs in a user
`logout()`	Logs out a user
`create_user()`	Creates a user with password hashing
`check_password()`	Checks a password
`set_password()`	Sets a new password hash

`get_user_model()`	Retrieves the configured User model
`get_username()`	Returns the user's username
`is_authenticated`	Indicates whether the request user is authenticated
`has_perm()`	Checks a specific permission
`has_perms()`	Checks multiple permissions
`groups.add()`	Adds a user to groups
`user_permissions.add()`	Assigns permissions directly

---

8. Important Interview Questions

Q1. What is authentication in Django?

Authentication is the process of verifying a user's identity using credentials such as username and password.

Q2. What is the difference between authentication and authorization?

Authentication verifies who the user is, while authorization determines what the user is allowed to do.

Q3. What is Django's default User model?

Django provides a built-in User model through `django.contrib.auth.models.User`.

Q4. What is the purpose of `authenticate()`?

It verifies the supplied credentials using configured authentication backends and returns a User object if authentication succeeds.

Q5. What is the difference between `authenticate()` and `login()`?

`authenticate()` verifies credentials, while `login()` associates the authenticated user with the current session.

Q6. What is UserCreationForm?

It is a built-in Django form used to register new users with username and password fields.

Q7. What is AuthenticationForm?

It is a built-in Django form used to authenticate existing users using their credentials.

Q8. What is password hashing?

Password hashing transforms a password into a salted hash so the original password is not stored directly in the database.

Q9. What is an authentication backend?

An authentication backend defines how Django verifies a user's identity.

Q10. What is ModelBackend?

It is Django's default authentication backend, which authenticates users against the configured User model.

Q11. What is a Django session?

A session maintains information about a user across multiple HTTP requests.

Q12. What is the purpose of AuthenticationMiddleware?

It makes the current user available through `request.user`.

Q13. What is the purpose of SessionMiddleware?

It makes session data available through `request.session`.

Q14. What is the difference between `is_staff` and `is_superuser`?

`is_staff` allows access to the Django admin site, subject to admin permissions.
`is_superuser` grants all permissions.

Q15. What are Django Groups?

Groups organize users and allow multiple users to share the same permissions.

Q16. What is `login_required`?

It is a decorator that restricts a view to authenticated users.

Q17. What is `permission_required`?

It is a decorator that restricts access to users who have a specified permission.

Q18. What is the difference between a User model and an authentication backend?

The User model stores account information, while the authentication backend defines how credentials are verified.

---

9. Final Revision

Topic	Main concepts
Authentication overview	Authentication, authorization, middleware
User model	User fields, ORM, password hashing
Authentication backends	ModelBackend, authenticate(), custom backends
Registration	UserCreationForm, form.save()
Login	AuthenticationForm, form.get_user(), login()
Logout	logout()
Sessions	request.session, cookies, SessionMiddleware
Permissions	has_perm(), default permissions
Groups	Shared permissions
Page protection	login_required, permission_required

Core concept to remember:

- User model → Stores user information.
- Authentication backend → Verifies credentials.
- Authentication form → Collects and validates login information.
- Registration form → Creates new accounts.
- Session → Maintains the logged-in state.
- Permissions → Control what users can do.
- Groups → Allow multiple users to share permissions.

End of Django Authentication Notes.

Django and Django REST Framework — Complete Study Notes

Table of Contents

1. Introduction to Django
2. Django Architecture — MVT
3. Django Project and App Structure
4. Creating a Django Project and App
5. Important Django Configuration
6. URLs and URL Routing
7. Views and HTTP Requests
8. Django Templates
9. Template Inheritance
10. Django Forms
11. Django Models and Database
12. Migrations
13. Django Built-in User Model
14. Registration, Login and Logout
15. Authentication, Authorization and Sessions
16. CSRF Protection
17. Authentication Middleware and Decorators
18. Introduction to REST APIs
19. Django REST Framework
20. Serializers
21. API Views and CRUD Operations
22. HTTP Methods and Status Codes
23. API Authentication
24. Session Authentication in DRF
25. Permissions
26. API Documentation and Swagger
27. Postman API Testing
28. API Keys
29. Deployment Fundamentals
30. AWS EC2 Deployment
31. Gunicorn, Nginx and Systemd
32. Environment Variables and Production Security
33. Common Errors and Solutions
34. Important Interview Questions
35. Topics to Practise Next

---

1. Introduction to Django

What is Django?

Django is a high-level Python web framework used to build web applications quickly and securely.

It provides built-in functionality for:

- URL routing
- Database management
- Authentication
- Authorization
- Forms and validation
- HTML templates
- Security
- Admin panel
- Session management

Instead of building every feature from scratch, Django provides reusable components.

Why do we use Django?

1. **Fast development:** Provides built-in features that reduce development time.
2. **Security:** Includes protection against common attacks.
3. **Scalability:** Supports applications that grow in complexity.
4. **ORM:** Allows interaction with databases using Python.
5. **Authentication:** Provides built-in user registration and authentication tools.
6. **Reusable applications:** Applications can be separated into independent Django apps.

What is a framework?

A framework is a collection of tools, libraries, rules and predefined structures that help developers build applications.

A framework provides a structure so developers don't have to write everything from scratch.

---

2. Django Architecture — MVT

Django follows the MVT architecture.

MVT stands for:

- Model
- View
- Template

2.1 Model

The Model represents the data and database structure.

It defines:

- Database fields
- Data types
- Relationships
- Database operations

Example:

```
None
from django.db import models


class Student(models.Model):

    name = models.CharField(max_length=100)

    age = models.IntegerField()

    email = models.EmailField()
```

Django ORM converts this model into a database table.

2.2 View

The View contains the application logic.

It receives requests, processes information and returns responses.

Example:

```
None
from django.shortcuts import render


def home(request):

    return render(request, 'home.html')
```

2.3 Template

The Template represents the presentation layer, usually HTML.

Example:

```
None
<h1>Welcome to Django</h1>
```

How MVT works

1. A user sends a request through a browser.
2. Django checks the URL configuration.
3. The URL routes the request to the appropriate View.
4. The View interacts with the Model if necessary.
5. The View returns a Template or another response.
6. Django sends the response to the browser.

---

3. Django Project and App Structure

What is a Django project?

A Django project is the entire web application.

It contains the main configuration and coordinates different applications.

What is a Django app?

An app is a specific module that performs a particular functionality.

For example:

- `login_app` — registration and login.
- `api` — API endpoints.
- `products` — product management.
- `students` — student management.

A single Django project can contain multiple apps.

Difference between project and app

Project	App
Represents the entire website	Represents a specific functionality
Contains project configuration	Contains application logic
Can contain multiple apps	Belongs to a project
Has the main settings.py	Usually has models.py, views.py and urls.py

Common Django project structure

```
None
```

```
django_project/
│
├── manage.py
│
```

```
├── django_project/
|   ├── __init__.py
|   ├── settings.py
|   ├── urls.py
|   ├── asgi.py
|   └── wsgi.py
|
├── api/
|   ├── migrations/
|   ├── models.py
|   ├── views.py
|   ├── serializers.py
|   ├── urls.py
|   └── admin.py
|
├── login_app/
|   ├── migrations/
|   ├── views.py
|   ├── urls.py
|   └── ...
|
```

```
├── templates/
├── db.sqlite3
└── ...
```

Important files

manage.py

Used to execute Django management commands.

Examples:

```
None
python manage.py runserver

python manage.py migrate

python manage.py makemigrations

python manage.py createsuperuser
```

settings.py

Contains project configuration.

Examples:

- Installed apps
- Middleware
- Database configuration
- Templates
- Static files
- Security settings

urls.py

Defines URL routes.

views.py

Contains request-handling logic.

models.py

Defines database models.

serializers.py

Defines how model data is converted into and validated from API representations.

admin.py

Used to register models in the Django admin panel.

migrations/

Stores database schema change instructions.

---

4. Creating a Django Project and App

Install Django:

```
None
pip install django
```

Create a project:

```
None
django-admin startproject django_project
```

Move into the project directory:

```
None
cd django_project
```

Create an app:

```
None
python manage.py startapp login_app
```

Run the development server:

```
None
python manage.py runserver
```

Default development address:

```
None
http://127.0.0.1:8000/
```

Stop the server using `CTRL + C`.

Registering an app

Open `settings.py`.

Add the app to `INSTALLED_APPS`:

```
None
INSTALLED_APPS = [

    'django.contrib.admin',

    'django.contrib.auth',

    'django.contrib.contenttypes',

    'django.contrib.sessions',

    'django.contrib.messages',

    'django.contrib.staticfiles',
```

```
    'rest_framework',


    'api',
    'login_app',
]
```

Why?

Django needs to know which applications belong to the project so it can load their models, configurations and other functionality.

---

5. Important Django Configuration

5.1 INSTALLED_APPS

Contains all Django applications enabled in the project.

Important built-in applications:

Application	Purpose
django.contrib.admin	Admin panel
django.contrib.auth	Authentication and users
django.contrib.contenttypes	Content type framework

django.contrib.sessions	Session management
django.contrib.messages	Temporary messages
django.contrib.staticfiles	Static file management

5.2 MIDDLEWARE

Middleware processes requests and responses globally.

It can perform operations before a request reaches a View and after a View generates a response.

Important middleware:

```
None
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

Important middleware explained

SessionMiddleware

Enables session management.

AuthenticationMiddleware

Makes the current authenticated user available through `request.user`.

CsrfViewMiddleware

Checks CSRF tokens on protected requests.

SecurityMiddleware

Provides various security-related protections.

5.3 DATABASES

Django uses a database to store application data.

Default SQLite configuration:

```
None
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Other supported databases include:

- PostgreSQL
- MySQL
- Oracle

5.4 Environment variables

Environment variables store configuration outside the source code.

For example, instead of writing a secret key directly:

```
None
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
```

For DEBUG:

```
None
DEBUG = os.environ.get("DJANGO_DEBUG") == "True"
```

For allowed hosts:

```
None
ALLOWED_HOSTS = os.environ.get(

    "DJANGO_ALLOWED_HOSTS", ""

).split(",")
```

Why use environment variables?

- Protect secret values.
- Separate development and production configurations.
- Avoid committing credentials to GitHub.
- Make deployment configuration easier.

---

6. URLs and URL Routing

URL routing connects a requested URL to a Django View.

Example:

```
None
from django.urls import path

from . import views
```

```
urlpatterns = [
    path('', views.home, name='home'),
    path('login/', views.login, name='login'),
    path('register/', views.register, name='register'),
]
```

Understanding path()

```
None
path('login/', views.login, name='login')
```

- `login/` — URL pattern.
- `views.login` — View that handles the request.
- `name='login'` — Name used to refer to this URL elsewhere.

include()

`include()` allows the main project URL configuration to include another app's URL configuration.

Main project `urls.py`:

```
None
from django.urls import path, include

urlpatterns = [
    path('', include('login_app.urls')),
```

```
    path('api/', include('api.urls')),
]
```

This makes the project easier to organize.

URL names

Instead of hardcoding URLs in templates:

```
None
<a href="/login/">Login</a>
```

Use:

```
None
<a href="{% url 'login' %}">Login</a>
```

This is useful because URLs can change without requiring every template to be updated.

---

7. Views and HTTP Requests

A View is a Python function or class that receives a request and returns a response.

Example:

```
None
from django.http import HttpResponse


def home(request):
    return HttpResponse("Welcome")
```

What is request?

`request` is an object containing information about the incoming HTTP request.

It provides:

- Request method
- Submitted form data
- User information
- Cookies
- Session information
- Headers

GET request

GET is commonly used to retrieve information.

```
None
def home(request):

    if request.method == 'GET':

        return HttpResponse("This is a GET request")
```

POST request

POST is commonly used to submit data.

```
None
def register(request):

    if request.method == 'POST':

        username = request.POST.get('username')
```

request.POST

Contains submitted form data from a POST request.

request.GET

Contains query parameters from a GET request.

Example:

```
None
/students/?name=Rashik
```

Access it using:

```
None
name = request.GET.get('name')
```

render()

Used to render an HTML template and return an HTTP response.

```
None
return render(request, 'home.html')
```

redirect()

Redirects the user to another URL.

```
None
return redirect('login')
```

HttpResponse()

Returns a direct HTTP response.

```
None
return HttpResponse("Hello")
```

---

8. Django Templates

Templates are HTML files used to display information.

Example:

```
None
def home(request):

    return render(request, 'home.html')
```

Passing data to templates

```
None
def home(request):

    context = {

        'username': 'Rashik',

        'age': 18

    }


    return render(request, 'home.html', context)
```

HTML:

```
None
<h1>Welcome {{ username }}</h1>

<p>Age: {{ age }}</p>
```

Django Template Language (DTL)

Django Template Language allows dynamic content inside HTML.

Variables

```
None

{{ username }}
```

If condition

```
None

{% if user.is_authenticated %}

    <p>Welcome {{ user.username }}</p>

{% else %}

    <p>Please log in</p>

{% endif %}
```

For loop

```
None

{% for student in students %}

    <p>{{ student.name }}</p>

{% endfor %}
```

URL tag

```
None

<a href="{% url 'login' %}">Login</a>
```

CSRF token

```
None

{% csrf_token %}
```

---

9. Template Inheritance

Template inheritance allows multiple HTML pages to share a common structure.

Instead of repeating the same navbar, header and footer on every page, we create a base template.

base.html

```
None
<!DOCTYPE html>

<html>

<head>

    <title>{% block title %}My Website{% endblock %}</title>

</head>

<body>


    <header>

        <h1>My Django Application</h1>

    </header>


    <main>

        {% block content %}

        {% endblock %}

    </main>
```

```
</body>

</html>
```

home.html

```
None
{% extends 'base.html' %}


{% block title %}

Home

{% endblock %}


{% block content %}

<h2>Welcome to Home Page</h2>

{% endblock %}
```

Important template inheritance tags

Tag	Purpose
extends	Inherits another template
block	Defines a section that child templates can override

endblock	Ends a block

Benefits:

- Avoids repeated HTML.
- Makes maintenance easier.
- Creates a consistent layout.
- Makes large projects easier to manage.

---

10. Django Forms

Forms are used to collect user input.

Django provides built-in form handling and validation.

There are two important types.

10.1 forms.Form

Used to create a form by defining fields manually.

Example:

```
None
from django import forms


class StudentForm(forms.Form):

    name = forms.CharField(max_length=100)

    age = forms.IntegerField()

    email = forms.EmailField()
```

10.2 forms.ModelForm

A ModelForm creates a form based on an existing Django model.

Example:

```
None
from django import forms

from .models import Student


class StudentForm(forms.ModelForm):


    class Meta:

        model = Student

        fields = '__all__'
```

Difference:

forms.Form	forms.ModelForm
Fields are manually defined	Fields are based on a model
Does not automatically save to a model	Provides model-saving functionality
Useful for general forms	Useful for database-backed forms

Form validation

Example:

```
None
if form.is_valid():

    form.save()
```

`is_valid()` checks whether submitted information satisfies the form's validation rules.

For a normal Django Form:

```
None
if form.is_valid():

    data = form.cleaned_data
```

cleaned_data

Contains validated and cleaned form values.

Example:

```
None
name = form.cleaned_data['name']
```

form.save()

Used with ModelForm or suitable model-backed forms to save information to the database.

Displaying forms

```
None
<form method="POST">

    {% csrf_token %}

    {{ form.as_p }}


    <button type="submit">Submit</button>
```

```
</form>
```

`form.as_p` renders each form field inside a paragraph element.

Other rendering options include `form.as_table` and `form.as_ul`.

---

11. Django Models and Database

Models define how data is stored in a database.

Example:

```
None
from django.db import models


class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
    created_at = models.DateTimeField(auto_now_add=True)
```

Common model fields

Field	Purpose
CharField	Short text

TextField	Long text
IntegerField	Integer values
FloatField	Decimal/floating-point values
BooleanField	True or False
EmailField	Email validation
DateField	Date
DateTimeField	Date and time
ForeignKey	Many-to-one relationship
OneToOneField	One-to-one relationship
ManyToManyField	Many-to-many relationship

What is Django ORM?

ORM stands for Object-Relational Mapping.

It allows developers to interact with database records using Python objects instead of writing SQL for every operation.

Example:

```
None
Student.objects.all()
```

This retrieves all Student records.

Create

```
None
Student.objects.create(
    name="Rashik",
    age=18,
    email="rashik@example.com"
)
```

Read

```
None
Student.objects.all()
```

Filter

```
None
Student.objects.filter(age=18)
```

Get

```
None
Student.objects.get(id=1)
```

`get()` expects exactly one matching record. It raises an exception if no record or multiple records are found.

Update

```
None
student = Student.objects.get(id=1)


student.name = "New Name"

student.save()
```

Delete

```
None
student = Student.objects.get(id=1)

student.delete()
```

---

12. Migrations

Migrations are used to apply model structure changes to the database.

Whenever we create or modify a model, we generally need migrations.

makemigrations

```
None
python manage.py makemigrations
```

Creates migration files based on model changes.

migrate

```
None
python manage.py migrate
```

Applies migrations to the database.

showmigrations

```
None

python manage.py showmigrations
```

Displays migration status.

Difference

makemigrations	migrate
Creates migration files	Applies migration files
Detects model changes	Changes the database schema
Does not itself apply the changes	Executes the database changes

---

13. Django Built-in User Model

Django provides a built-in User model through `django.contrib.auth`.

It includes common user information such as:

- Username
- Password hash
- Email
- First name
- Last name
- Staff status

- Superuser status
- Active status

We do not need to create a new user model for basic registration and login.

UserCreationForm

Django provides `UserCreationForm` for creating users.

```
None
from django.contrib.auth.forms import UserCreationForm
```

It handles:

- Username validation
- Password input
- Password confirmation
- Password validation
- User creation

Example:

```
None
form = UserCreationForm()
```

After validation:

```
None
if form.is_valid():

    form.save()
```

Passwords are stored as hashes, not as plain text.

---

14. Registration, Login and Logout

This is the authentication system implemented in your `login_app`.

14.1 Registration

None

```
from django.shortcuts import render, redirect

from django.contrib.auth.forms import UserCreationForm


def register(request):


    if request.method == 'POST':

        form = UserCreationForm(request.POST)


        if form.is_valid():

            form.save()

            return redirect('login')


    else:

        form = UserCreationForm()


    return render(request, 'register.html', {'form': form})
```

How it works

1. The user opens the registration page.
2. Django displays `UserCreationForm`.
3. The user submits the form.
4. Django receives the POST request.
5. `UserCreationForm` validates the input.

6. `form.save()` creates the user.
7. The user is redirected to login.

register.html

```
None
<h1>Register</h1>


<form method="POST">
    {% csrf_token %}
    {{ form.as_p }}

    <button type="submit">Register</button>
</form>
```

14.2 Login

Django provides `AuthenticationForm` to validate login credentials.

```
None
from django.contrib.auth.forms import AuthenticationForm
from django.contrib.auth import login as auth_login


def login(request):

    if request.user.is_authenticated:
        return redirect('home')
```

```
    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)

        if form.is_valid():
            user = form.get_user()

            auth_login(request, user)

            return redirect('home')

    else:
        form = AuthenticationForm()

    return render(request, 'login.html', {'form': form})
```

How it works

1. The user enters a username and password.
2. The form receives the submitted information.
3. `AuthenticationForm` validates the credentials using Django's authentication system.
4. `form.get_user()` returns the authenticated user.
5. `auth_login()` logs the user into the application.
6. Django associates the user with the session.
7. The user is redirected to the home page.

What is authenticate()?

`authenticate()` checks the supplied credentials against Django's configured authentication backends.

Example:

```
None
from django.contrib.auth import authenticate


user = authenticate(

    request,

    username='rashik',

    password='password123'

)


if user is not None:

    print("Valid credentials")

else:

    print("Invalid credentials")
```

Important distinction:

- `authenticate()` checks credentials and returns a user or `None`.
- `login()` establishes the authenticated session.

14.3 Logout

```
None
from django.contrib.auth import logout as auth_logout
```

```
def logout_view(request):


    if request.method == 'POST':
        auth_logout(request)
        return redirect('login')


    return redirect('home')
```

HTML:

```
None
<form method="POST" action="{% url 'logout' %}">
    {% csrf_token %}
    <button type="submit">Logout</button>
</form>
```

`auth_logout(request)` ends the current authenticated session.

Logout is submitted using POST rather than a normal GET link.

---

15. Authentication, Authorization and Sessions

Authentication

Authentication verifies who a user is.

Example:

- Username and password.
- Session login.
- Token authentication.

Question answered: **Who are you?**

Authorization

Authorization determines what an authenticated user is allowed to do.

Example:

- A normal user can view records.
- An administrator can delete records.
- A staff member can access the admin panel.

Question answered: **What are you allowed to do?**

Sessions

A session stores information about a user's interaction with a website across multiple requests.

HTTP is stateless, which means each HTTP request is independent by default.

Sessions help Django remember authenticated users.

How Django sessions work

1. A user submits valid credentials.
2. Django authenticates the user.
3. Django creates or updates the session.
4. Django sends a session cookie to the browser.
5. The browser sends that cookie with subsequent requests.
6. Django identifies the session and makes the authenticated user available through `request.user`.

The session data is stored on the server, while the browser normally stores the session identifier in a cookie.

Session vs Cookie

Session	Cookie
Stores session data on the server by default	Stores data in the browser
Identified using a session key	Stored directly in the browser
Can be used to maintain login state	Can carry a session identifier

---

16. CSRF Protection

CSRF stands for Cross-Site Request Forgery.

It is an attack where a malicious website attempts to make a user's browser submit an unwanted request to another website where the user is already authenticated.

Django provides CSRF protection.

How it works

1. Django generates a CSRF token.
2. The token is included in a form.
3. The browser submits the form.
4. Django checks the submitted token.
5. If validation fails, Django rejects the request.

Example:

```
None
<form method="POST">

    {% csrf_token %}
```

```
    <button type="submit">Submit</button>
</form>
```

The CSRF token is especially important for POST requests using session authentication.

CSRF protection is not the same as authentication. Authentication identifies a user; CSRF protection helps verify that a protected request originated from an expected context.

---

17. Authentication Middleware and Decorators

AuthenticationMiddleware

It makes the current user available in the request object.

Example:

```
None
request.user
```

You can check:

```
None
request.user.is_authenticated
```

It returns:

- `True` if the user is authenticated.
- `False` if the user is anonymous.

login_required

Protects a View so only authenticated users can access it.

```
None
from django.contrib.auth.decorators import login_required


@login_required(login_url='login')

def home(request):

    return render(request, 'home.html')
```

If an unauthenticated user opens the page, Django redirects them to login.

is_authenticated

Can be used directly in a View or template.

Python:

```
None
if request.user.is_authenticated:

    print("Logged in")
```

Template:

```
None
{% if user.is_authenticated %}

    <p>Welcome {{ user.username }}</p>

{% else %}

    <p>Please log in</p>

{% endif %}
```

never_cache

Browsers may display cached pages when users press the Back button after logout.

Use `never_cache` on protected pages:

```
None
from django.views.decorators.cache import never_cache

from django.contrib.auth.decorators import login_required



@never_cache

@login_required(login_url='login')

def home(request):

    return render(request, 'home.html')
```

This instructs browsers and caches not to store the response.

The important security check is still `login_required`; `never_cache` helps prevent stale protected content from being displayed.

---

18. Introduction to REST APIs

What is an API?

API stands for Application Programming Interface.

An API allows two software applications to communicate with each other.

Example:

A frontend application sends a request to a Django API. Django processes the request and returns data, commonly in JSON format.

What is a REST API?

REST stands for Representational State Transfer.

REST is an architectural style for designing network APIs.

Common REST principles include:

- Client-server separation
- Stateless communication
- Resource-based URLs
- Standard HTTP methods
- Representations such as JSON

Example:

```
None
/api/students/
```

This endpoint represents a collection of students.

What is JSON?

JSON stands for JavaScript Object Notation.

It is a lightweight data format commonly used to exchange information between applications.

Example:

```
None
{
    "id": 1,
    "name": "Rashik",
    "age": 18,
    "email": "rashik@example.com"
}
```

HTML vs JSON

HTML	JSON
Used to display web pages	Used to exchange structured data
Contains presentation markup	Contains data
Commonly rendered by browsers	Commonly processed by applications
Used in traditional Django templates	Commonly used in REST APIs

---

19. Django REST Framework

What is DRF?

DRF stands for Django REST Framework.

It is a toolkit built on Django for creating Web APIs.

It provides:

- Serializers
- API Views
- Request and Response classes
- Authentication
- Permissions
- Pagination
- Filtering
- Browsable API
- API documentation integration

Installing DRF

```
None
pip install djangorestframework
```

Add it to `INSTALLED_APPS`:

```
None
INSTALLED_APPS = [
    # Django apps
    'rest_framework',
    'api',
]
```

Difference between Django and DRF

Django	Django REST Framework
General web framework	Toolkit for building APIs
Commonly returns HTML	Commonly returns JSON
Uses Django Views	Provides API Views and ViewSets
Uses Django Forms	Uses Serializers for API validation and representation
Supports authentication	Extends authentication for APIs

---

20. Serializers

A Serializer converts complex data, such as Django model instances, into Python representations that can be rendered as JSON.

It also validates incoming data before it is saved.

Why do we need serializers?

Django models contain Python objects, but an API commonly exchanges JSON.

Serializers bridge that gap.

They perform two important operations:

1. Serialization — converting model data into a representation suitable for API responses.
2. Deserialization and validation — checking incoming data and preparing it for saving.

ModelSerializer

A ModelSerializer automatically generates fields based on a Django model.

Example:

```
None
from rest_framework import serializers

from .models import Student


class StudentSerializer(serializers.ModelSerializer):


    class Meta:

        model = Student

        fields = '__all__'
```

Understanding the code

`serializers.ModelSerializer` means the serializer is based on a Django model.

`class Meta` contains configuration.

`model = Student` specifies the model.

`fields = '__all__'` includes all model fields.

You can also specify selected fields:

```
None
fields = ['id', 'name', 'age', 'email']
```

Serializer validation

```
None
serializer = StudentSerializer(data=request.data)


if serializer.is_valid():

    serializer.save()
```

is_valid()

Checks whether the submitted data satisfies serializer validation rules.

validated_data

Contains the validated data after successful validation.

save()

Saves the validated data, generally by creating or updating a model instance.

serializer.data

Contains the serialized representation used in the API response.

raise_exception=True

```
None
```

```
serializer.is_valid(raise_exception=True)
```

Automatically raises a validation exception if the data is invalid. DRF handles the corresponding API error response.

---

21. API Views and CRUD Operations

CRUD stands for:

- Create
- Read
- Update
- Delete

We implemented a Student API using Django REST Framework.

Example Student API View

```
None
```

```
from rest_framework.decorators import api_view

from rest_framework.response import Response

from rest_framework import status


from .models import Student

from .serializers import StudentSerializer




@api_view(['GET', 'POST'])
```

```
def student_list(request):

    if request.method == 'GET':

        students = Student.objects.all()

        serializer = StudentSerializer(
            students,
            many=True
        )

        return Response(serializer.data)

    elif request.method == 'POST':

        serializer = StudentSerializer(
            data=request.data
        )

        if serializer.is_valid():
```

```
            serializer.save()

            return Response(
                serializer.data,
                status=status.HTTP_201_CREATED
            )

        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )
```

Understanding important components

`@api_view(['GET', 'POST'])`

Specifies which HTTP methods the API endpoint accepts.

`request.data`

Contains parsed incoming request data.

`Response()`

Creates an API response that DRF can render into the appropriate format.

`many=True`

Indicates that the serializer is handling multiple model instances rather than one object.

`status.HTTP_201_CREATED`

Indicates that a new resource was successfully created.

CRUD mapping

Operation	HTTP method	Purpose
Create	POST	Create a new record
Read	GET	Retrieve records
Update	PUT	Replace/update a resource
Partial update	PATCH	Update selected fields
Delete	DELETE	Delete a resource

Example API endpoints

```
None
GET     /api/students/
POST    /api/students/
GET     /api/students/1/
PUT     /api/students/1/
PATCH   /api/students/1/
DELETE  /api/students/1/
```

The exact methods supported depend on how the Views and URL routes are implemented.

Example single-student view

None

```
@api_view(['GET', 'PUT', 'PATCH', 'DELETE'])

def student_detail(request, pk):


    try:

        student = Student.objects.get(pk=pk)


    except Student.DoesNotExist:

        return Response(

            {'error': 'Student not found'},

            status=status.HTTP_404_NOT_FOUND

        )


    if request.method == 'GET':


        serializer = StudentSerializer(student)


        return Response(serializer.data)


    elif request.method in ['PUT', 'PATCH']:
```

```
        serializer = StudentSerializer(
            student,
            data=request.data,
            partial=request.method == 'PATCH'
        )

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)

        return Response(
            serializer.errors,
            status=status.HTTP_400_BAD_REQUEST
        )

    elif request.method == 'DELETE':

        student.delete()

        return Response(
            status=status.HTTP_204_NO_CONTENT
```

```
        )
```

---

22. HTTP Status Codes

Status codes tell the client what happened when a request was processed.

Status	Meaning
200	OK
201	Created
202	Accepted
204	No Content
400	Bad Request
401	Unauthorized / unauthenticated
403	Forbidden
404	Not Found

405 Method Not Allowed

500 Internal Server Error

Examples:

- A successful GET request commonly returns 200.
- A successful POST creating a record commonly returns 201.
- Invalid input commonly returns 400.
- A missing record commonly returns 404.
- A request using an unsupported HTTP method commonly returns 405.

Note: HTTP 401 and 403 have distinct meanings. The exact response for an unauthenticated request can depend on the configured authentication class.

---

23. API Authentication

API authentication verifies the identity of the client making a request.

Common methods include:

1. Session Authentication
2. Token Authentication
3. Basic Authentication
4. API Key Authentication
5. JWT Authentication

Django website authentication vs API authentication

Website authentication	API authentication
Commonly uses browser sessions	Can use sessions, tokens, API keys or JWT

Usually uses HTML login forms	Often uses API requests
Browser stores session cookie	Client sends configured credentials
CSRF protection applies to unsafe session-authenticated requests	Depends on the authentication method

Authentication and permissions work together.

Authentication identifies the requester; permissions determine whether the requester can perform an operation.

---

24. Session Authentication in DRF

You specifically explored username/password authentication using sessions.

DRF provides `SessionAuthentication`.

Example configuration:

```
None
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.SessionAuthentication',
    ],

    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
```

```
    ],
}
```

How session authentication works

1. The user logs into Django.
2. Django authenticates the credentials.
3. Django creates an authenticated session.
4. The browser receives a session cookie.
5. The browser sends the cookie with API requests.
6. DRF identifies the user from the session.
7. DRF checks the endpoint's permissions.

CSRF with SessionAuthentication

When using session authentication, unsafe requests such as POST, PUT, PATCH and DELETE require CSRF protection for authenticated users.

For a browser-based client, the CSRF token must be obtained and sent appropriately.

Example request header:

```
None
X-CSRFToken: <csrf-token>
```

A session cookie alone is not a replacement for the CSRF token.

Login and logout endpoints

A traditional Django login form can establish the session used by DRF.

This is one way to connect a Django website's authentication system to a session-authenticated API.

---

25. Permissions

Permissions determine whether a request should be allowed.

DRF provides built-in permission classes.

AllowAny

Allows any user to access the endpoint.

```
None
from rest_framework.permissions import AllowAny
```

IsAuthenticated

Only authenticated users can access the endpoint.

```
None
from rest_framework.permissions import IsAuthenticated
```

IsAdminUser

Allows staff users to access the endpoint.

```
None
from rest_framework.permissions import IsAdminUser
```

IsAuthenticatedOrReadOnly

Authenticated users can perform permitted write operations, while unauthenticated users can access read-only operations.

```
None
from rest_framework.permissions import IsAuthenticatedOrReadOnly
```

Applying permissions

```
None
from rest_framework.decorators import (
    api_view,
    permission_classes
)

from rest_framework.permissions import IsAuthenticated



@api_view(['GET'])
@permission_classes([IsAuthenticated])
def student_list(request):
    ...
```

Authentication vs Permission

Authentication	Permission
Identifies the user	Checks access rights
Verifies credentials	Allows or denies operations
Answers who is making the request	Answers what they can do

26. API Documentation and Swagger

What is API documentation?

API documentation explains how developers can use an API.

It describes:

- Available endpoints
- HTTP methods
- Required parameters
- Request body
- Response structure
- Authentication requirements
- Status codes
- Error responses

What is a schema?

A schema is a structured description of an API.

It defines:

- Available endpoints
- Supported HTTP methods
- Request fields
- Data types
- Required fields
- Response structures
- Authentication methods

A schema describes the API's contract; it does not itself validate the incoming data.

Serializers perform data validation in DRF.

What is OpenAPI?

OpenAPI is a standard specification for describing HTTP APIs.

It allows tools to understand an API's structure without needing to read its entire implementation.

What is Swagger?

Swagger is a collection of tools used to work with OpenAPI specifications.

Swagger UI provides an interactive interface for exploring and testing API endpoints.

drf-spectacular

`drf-spectacular` is a popular package used to generate OpenAPI schemas for Django REST Framework APIs.

Install:

```
None
pip install drf-spectacular
```

Add to `INSTALLED_APPS`:

```
None
'drf_spectacular',
```

Configure:

```
None
REST_FRAMEWORK = {

    'DEFAULT_SCHEMA_CLASS':

        'drf_spectacular.openapi.AutoSchema',

}
```

Add schema and Swagger routes in your project URL configuration:

```
None
from drf_spectacular.views import (
    SpectacularAPIView,
    SpectacularSwaggerView
)

urlpatterns = [
    path(
        'api/schema/',
        SpectacularAPIView.as_view(),
        name='schema'
    ),

    path(
        'api/docs/',
        SpectacularSwaggerView.as_view(
            url_name='schema'
        ),
        name='swagger-ui'
    ),
]
```

Then visit:

```
None
/api/docs/

/api/schema/
```

Swagger UI can display endpoints and allow interactive requests, depending on the authentication and permissions configured.

---

27. Postman API Testing

Postman is a tool used to send HTTP requests and test APIs.

You can test:

- GET
- POST
- PUT
- PATCH
- DELETE

Example GET request

```
None

GET http://127.0.0.1:8000/api/students/
```

Example POST request

Select:

- Method: POST
- Body: raw
- Format: JSON

Example body:

```
None
{
    "name": "Rashik",
    "age": 18,
    "email": "rashik@example.com"
}
```

Example PUT request

```
None
PUT /api/students/1/
```

Send the complete updated resource.

Example PATCH request

```
None
PATCH /api/students/1/
```

Send only the fields that need to change.

Example DELETE request

```
None
DELETE /api/students/1/
```

Headers

Headers provide additional information about an HTTP request.

Example:

```
None
Content-Type: application/json

Accept: application/json
```

For authenticated session-based requests, you may also need:

- Session cookie
- CSRF token

Postman can maintain cookies, but the CSRF requirements must still be satisfied.

---

28. API Keys

An API key is a credential used to identify or authorize an API client.

It is different from a Django username/password login.

Common uses:

- Restrict access to an API.
- Identify an application.
- Control API usage.
- Support key revocation.

An API key is usually sent through an HTTP header, depending on the implementation.

For example:

```
None
Authorization: Api-Key YOUR_API_KEY
```

The exact header format depends on the API-key package or custom implementation.

An API key should be kept secret and should not be placed in public frontend code.

API keys are not a complete substitute for user authentication and permissions when an application needs individual user identities.

---

29. Deployment Fundamentals

Deployment means making an application available on a server so other people can access it over a network.

A local Django project runs on your own machine.

A deployed project runs on a remote server.

Development vs Production

Development	Production
Used while building the application	Used by real users
DEBUG is usually True	DEBUG must be False
Often uses runserver	Uses a production application server
Local development database may be used	Production database should be configured appropriately
Development settings	Secure production settings

Important deployment components

- Server: Hosts the application.
- Python: Runs the Django code.
- Virtual environment: Isolates Python dependencies.
- Gunicorn: Runs Django as a WSGI application.
- Nginx: Acts as a web server and reverse proxy.
- Systemd: Manages the application service.
- Environment variables: Store deployment configuration.
- Security groups/firewall: Control network access.

---

30. AWS EC2 Deployment

You deployed your Django REST API using AWS EC2.

Your deployed API endpoint:

http://16.176.3.13/api/students/

What is AWS?

AWS stands for Amazon Web Services.

It provides cloud computing services such as:

- Virtual servers
- Databases
- Storage
- Networking
- Security
- Monitoring

What is EC2?

EC2 stands for Elastic Compute Cloud.

It provides virtual machines in the AWS cloud.

You can install software, run applications and configure the server.

What is an AMI?

AMI stands for Amazon Machine Image.

It is a template used to launch EC2 instances.

It contains information such as the operating system and preconfigured software.

What is a Security Group?

A Security Group acts as a virtual firewall for an EC2 instance.

It controls inbound and outbound network traffic.

Examples of inbound rules:

Port	Purpose
22	SSH
80	HTTP
443	HTTPS

Only expose ports that are needed, and restrict SSH access to trusted IP addresses where possible.

What is an SSH key pair?

An SSH key pair allows secure access to an EC2 instance.

You used a `.pem` private key to connect to your Ubuntu server.

Example:

```
None
ssh -i student-api-key.pem ubuntu@YOUR_PUBLIC_IP
```

The private key must be kept secure.

What is a public IP?

A public IP allows a server to be reached over the internet.

Your EC2 instance used:

```
None
16.176.3.13
```

An Elastic IP is a static public IPv4 address that can be associated with an EC2 instance.

A normal automatically assigned public IP can change after certain instance lifecycle operations.

---

31. Gunicorn, Nginx and Systemd

Gunicorn

Gunicorn stands for Green Unicorn.

It is a Python WSGI HTTP server used to run Django applications in production.

Why not use runserver?

Django's development server is intended for development, not production.

Gunicorn provides a production-oriented way to serve Django's WSGI application.

Example:

```
None
gunicorn django_project.wsgi:application
```

WSGI

WSGI stands for Web Server Gateway Interface.

It is a standard interface between Python web applications and WSGI servers.

Django provides a WSGI application through:

```
None
django_project/wsgi.py
```

Nginx

Nginx is a web server and reverse proxy.

It can:

- Receive incoming HTTP requests.
- Serve static files.
- Forward application requests to Gunicorn.
- Handle reverse-proxy configuration.
- Support HTTPS termination.

Systemd

Systemd is a Linux system and service manager.

You used a systemd service to keep your Django application running.

Your service was named:

```
None
django-api
```

Check service status:

```
None
sudo systemctl status django-api
```

Restart:

```
None
sudo systemctl restart django-api
```

Stop:

```
None
sudo systemctl stop django-api
```

Start:

```
None
sudo systemctl start django-api
```

View logs:

```
None
sudo journalctl -u django-api -f
```

How the deployment request flows

1. A client sends a request to your server.
2. Nginx receives the request.
3. Nginx forwards the application request to Gunicorn.
4. Gunicorn communicates with Django through WSGI.
5. Django processes the request.
6. Django returns a response.
7. Gunicorn passes the response back to Nginx.
8. Nginx sends it to the client.

---

32. Environment Variables and Production Security

Your deployment used an environment file:

```
None
/etc/django-api.env
```

It contained deployment configuration such as the Django secret key and debug/host settings.

Why use environment variables?

They help keep sensitive configuration out of the source code.

Example:

```
None
SECRET_KEY = os.environ["DJANGO_SECRET_KEY"]
```

Production configuration should include:

```
None
DJANGO_DEBUG=False

DJANGO_ALLOWED_HOSTS=YOUR_PUBLIC_IP
```

The actual environment file should contain your real secret key, not a placeholder.

DEBUG

Development:

```
None
DEBUG = True
```

Production:

```
None
DEBUG = False
```

When DEBUG is enabled, detailed error information can be exposed.

ALLOWED_HOSTS

Django uses this setting to validate the host header of incoming requests.

Example:

```
None
ALLOWED_HOSTS = ['16.176.3.13']
```

Configure it using environment variables in production.

collectstatic

Collects static files from the project and installed applications into the configured static root.

Command:

```
None
python manage.py collectstatic
```

You ran this during deployment.

Requirements file

A requirements file records Python package dependencies.

Example:

```
None
Django

djangorestframework

gunicorn
```

Install dependencies using:

```
None
pip install -r requirements.txt
```

Migrations in production

Run:

```
None
python manage.py migrate
```

This applies database migrations on the server.

---

33. Common Errors and Solutions

1. ModuleNotFoundError

Example:

```
None
ModuleNotFoundError: No module named 'rest_framework'
```

Reason:

- Package is not installed in the active environment.
- Wrong virtual environment is activated.

Solution:

```
None
pip install djangorestframework
```

2. NoReverseMatch

Reason:

- A URL name used in `redirect()` or `{% url %}` does not exist.

Example:

```
None
return redirect('login')
```

Make sure your URL configuration includes:

```
None
path('login/', views.login, name='login')
```

3. OperationalError: no such table

Reason:

- Required database migrations have not been applied.

Solution:

```
None
python manage.py makemigrations

python manage.py migrate
```

4. CSRF verification failed

Possible reasons:

- Missing `{% csrf_token %}`.
- Missing or incorrect CSRF header.
- Session-based POST request without a valid CSRF token.

For HTML forms:

```
None
<form method="POST">

    {% csrf_token %}

</form>
```

5. 404 Not Found

Possible reasons:

- Incorrect URL.
- Missing URL pattern.
- Incorrect `include()` configuration.
- Resource does not exist.

6. 405 Method Not Allowed

Reason:

The API endpoint does not support the HTTP method being used.

For example, sending DELETE to a GET-only endpoint.

7. 403 Forbidden

Possible reasons:

- Permission denied.
- CSRF validation failed.
- User is authenticated but does not have permission.

8. Login page keeps appearing

Possible reasons:

- Invalid credentials.
- Session cookie is not being retained.
- Authentication middleware is missing.
- Login session is not being established.
- Protected page requires authentication.

Check:

```
None
request.user.is_authenticated
```

9. Home page appears after logout when pressing Back

The browser may display cached content.

Use:

```
None
from django.views.decorators.cache import never_cache


@never_cache
```

```
@login_required(login_url='login')

def home(request):

    return render(request, 'home.html')
```

The `login_required` decorator provides access control; `never_cache` helps prevent stale cached content from appearing.

---

34. Important Interview Questions

Django fundamentals

Q1. What is Django?

Django is a high-level Python web framework used to build secure and scalable web applications.

Q2. What is MVT architecture?

MVT stands for Model, View and Template. Model manages data, View handles application logic, and Template handles presentation.

Q3. What is the difference between a project and an app?

A project represents the entire application, while an app represents a particular functionality within the project.

Q4. What is ORM?

ORM stands for Object-Relational Mapping. It allows developers to interact with databases using Python objects instead of writing SQL manually.

Q5. What is middleware?

Middleware is a component that processes requests and responses globally.

Q6. What is the purpose of settings.py?

It contains project configurations such as installed apps, middleware, database settings, templates and security settings.

Q7. What is the purpose of urls.py?

It maps URL patterns to their corresponding Views.

Q8. What is the difference between render() and redirect()?

`render()` returns an HTTP response containing rendered template content. `redirect()` sends the client to another URL.

Q9. What is template inheritance?

It allows child templates to reuse and extend a common parent template.

Q10. What is the difference between Form and ModelForm?

Form defines fields manually, while ModelForm generates fields based on a Django model.

Authentication

Q11. What is authentication?

Authentication verifies the identity of a user.

Q12. What is authorization?

Authorization determines what an authenticated user can access or perform.

Q13. What is authenticate()?

It checks credentials using Django's configured authentication backends.

Q14. What is login()?

It establishes an authenticated session for the user.

Q15. What is logout()?

It ends the current authenticated session.

Q16. What is AuthenticationMiddleware?

It makes the current user available through `request.user`.

Q17. What is login_required?

A decorator that restricts a View to authenticated users.

Q18. What is CSRF protection?

It helps prevent malicious sites from submitting unauthorized requests using a user's authenticated browser.

Q19. What is a session?

A session allows a web application to remember information about a user across multiple requests.

Q20. What is an authentication backend?

It is a component that determines how Django authenticates users, such as checking username/password credentials.

Django REST Framework

Q21. What is DRF?

Django REST Framework is a toolkit for building Web APIs using Django.

Q22. What is a serializer?

A serializer converts complex data into API-friendly representations and validates incoming data.

Q23. What is ModelSerializer?

It is a serializer that automatically generates fields based on a Django model.

Q24. What is request.data?

It contains parsed request body data in DRF.

Q25. What is Response?

It is DRF's response class used to return API content and status information.

Q26. What is many=True?

It tells a serializer that it is handling multiple objects.

Q27. What is CRUD?

CRUD means Create, Read, Update and Delete.

Q28. What is the difference between PUT and PATCH?

PUT generally replaces a resource, while PATCH updates selected fields.

Q29. What is API authentication?

It verifies the identity of a client making an API request.

Q30. What are permissions?

Permissions determine whether an API request is allowed to access a particular resource or operation.

Q31. What is SessionAuthentication?

It is a DRF authentication class that uses Django's session framework to identify authenticated users.

Q32. What is API documentation?

It describes endpoints, methods, parameters, request formats, response formats and authentication requirements.

Q33. What is Swagger UI?

It is an interactive interface for exploring and testing an API described using OpenAPI.

Q34. What is an API key?

An API key is a credential used to identify or authorize an application or API client.

Deployment

Q35. What is AWS EC2?

EC2 is an AWS service that provides virtual computing instances.

Q36. What is Gunicorn?

Gunicorn is a production-oriented WSGI server for Python web applications.

Q37. What is Nginx?

Nginx is a web server and reverse proxy commonly used in front of application servers.

Q38. What is WSGI?

WSGI is a standard interface between Python web applications and WSGI servers.

Q39. Why do we use environment variables?

They keep configuration and sensitive values separate from application source code.

Q40. Why should DEBUG be False in production?

Because detailed debug information can expose sensitive application information.

---

35. Topics to Practise Next

These topics extend what you have already learned.

Django

1. Class-Based Views (CBVs)
2. Django generic views
3. Custom user models
4. Django groups and permissions
5. Password reset and email verification
6. File and image uploads
7. One-to-one, one-to-many and many-to-many relationships
8. Django signals
9. QuerySets and database optimization
10. Automated testing

Django REST Framework

1. APIView
2. GenericAPIView
3. Mixins
4. Generic views
5. ViewSets
6. Routers
7. Custom permissions
8. Token authentication
9. JWT authentication
10. Pagination
11. Filtering and searching

12. Throttling
13. Nested serializers
14. Custom serializer validation
15. API testing

Deployment

1. Domain configuration
2. HTTPS and SSL certificates
3. PostgreSQL deployment
4. Production logging
5. Database backups
6. Automated deployment
7. Monitoring and security

Final Summary

You have learned the foundations of building a Django application and a REST API.

Your practical work includes:

- Creating a Django project and applications.
- Configuring settings, middleware and URLs.
- Working with Views and Templates.
- Understanding template inheritance.
- Using Django Forms and ModelForms.
- Working with models, databases and migrations.
- Implementing registration and login using Django's built-in User model.
- Managing sessions and logout.
- Understanding CSRF protection.
- Protecting pages with authentication decorators.
- Creating a Student REST API.
- Using serializers for data conversion and validation.
- Implementing CRUD operations.
- Learning API authentication and permissions.
- Understanding session-authenticated APIs.
- Exploring OpenAPI and Swagger documentation.
- Testing APIs using Postman.
- Deploying a Django REST API on AWS EC2 using Ubuntu, Gunicorn, Nginx and systemd.

Your deployed API:

`http://16.176.3.13/api/students/`

This is your practical milestone: you have moved from running a Django application locally to making a REST API available on a cloud server.

Use these notes as your reference while continuing to practise Django and DRF.